Lab 5: Deep Q-Network and Deep Deterministic Policy Gradient

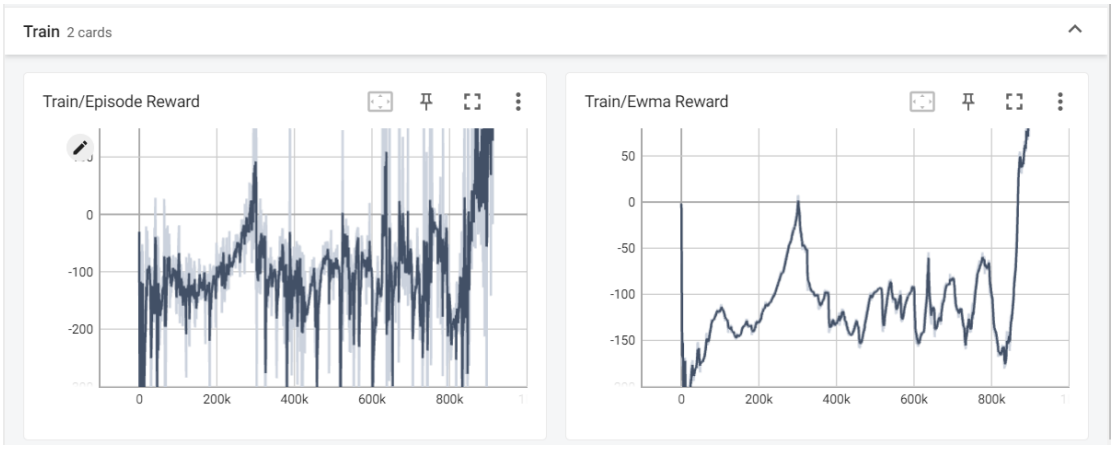
311351009 劉承霈

- Experimental Results

  Screenshot of tensorboard and testing results on:

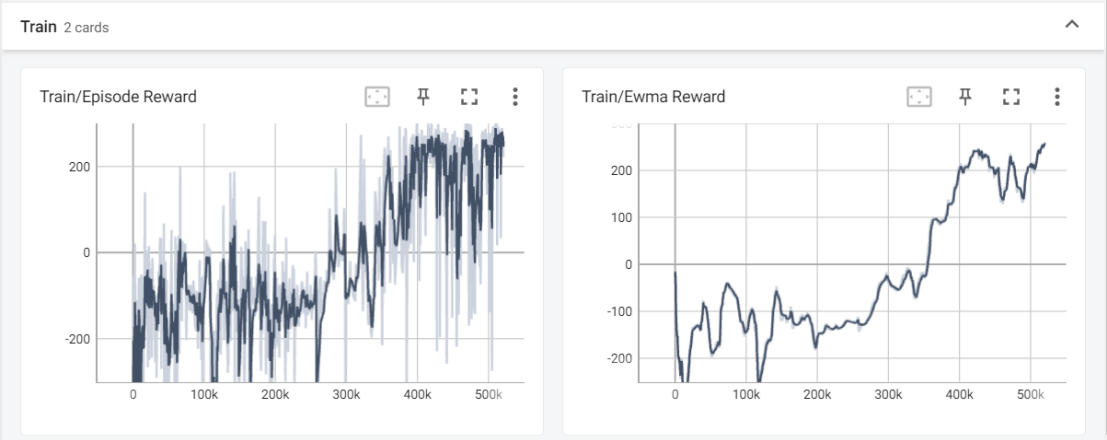
  - LunarLander-v2

```
pp037@ec037:~/DL/Lab5$ python3 dqn-example.py --test_only
Start Testing
episode 1: 217.29
episode 2: 252.41
episode 3: 252.07
episode 4: 252.00
episode 5: 306.98
episode 6: 237.87
episode 7: 292.14
episode 8: 269.23
episode 9: 293.93
episode 10: 237.97
Average Reward: 261.19
```



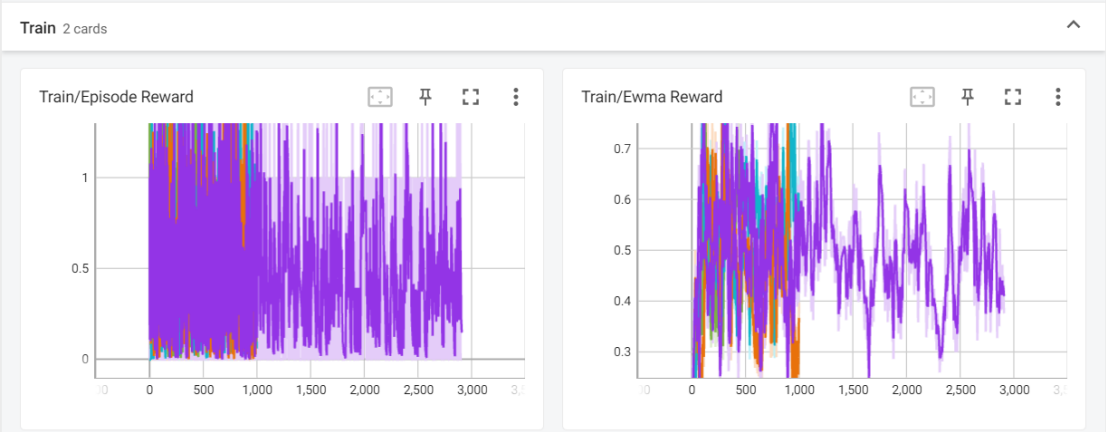
- LunarLanderContinuous-v2

```
pp037@ec037:~/DL/Lab5$ python3 ddpg-example.py --test_only
Start Testing
episode 1: 243.93
episode 2: 281.11
episode 3: 264.76
episode 4: 274.73
episode 5: 305.09
episode 6: 239.08
episode 7: 305.61
episode 8: 289.36
episode 9: 306.57
episode 10: 245.35
Average Reward: 275.56
```



- BreakoutNoFrameskip-v4

```
pp037@ec037:~/DL/Lab5$ python3 dqn_breakout_example.py --test_only
Start Testing
episode 1: 403.00
episode 2: 429.00
episode 3: 424.00
episode 4: 424.00
episode 5: 457.00
episode 6: 428.00
episode 7: 400.00
episode 8: 424.00
episode 9: 417.00
episode 10: 424.00
Average Reward: 423.00
```



- Questions
  - Describe the major implementation of both DQN and DDPG in detail.

    1. Implementation of Q network updating in DQN:

```
def _update_behavior_network(self, gamma):
    # sample a minibatch of transitions
    state, action, reward, next_state, done = self._memory.sample(
        self.batch_size, self.device)

    ## TODO ##
    q_value = self._behavior_net(state).gather(dim = 1, index = action.long())

    with torch.no_grad():
       q_next = self._target_net(next_state).max(dim = 1)[0].view(-1,1)
       q_target = reward + gamma * q_next * (1 - done)

    criterion = nn.MSELoss()
    loss = criterion(q_value, q_target)

    # optimize
    self._optimizer.zero_grad()
    loss.backward()
    nn.utils.clip_grad_norm_(self._behavior_net.parameters(), 5)
    self._optimizer.step()

def _update_target_network(self):
    '''update target network by copying from behavior network'''
    ## TODO ##
    self._target_net.load_state_dict(self._behavior_net.state_dict())
```

In the DQN model, there are two Q networks known as the target Q network and the behavior Q network. The behavior Q network is updated using Q values, specifically by adjusting it according to the difference between the predicted Q value of the next state and the actual Q value of that same state. Conversely, the target Q network is updated by duplicating the parameters from the behavior Q network.

2. Implementation and the gradient of actor updating in DDPG:

```
action = actor_net(state)
actor_loss = - critic_net(state, action).mean()
```

Since the gradient of the actor network can be expressed as the average of the outputs from the critic network, the implementation is carried out as shown above. It's important to note that a multiplication by -1 is applied to convert it into a gradient descent operation.

3. Implementation and the gradient of critic updating in DDPG:

```
q_value = critic_net(state, action)
with torch.no_grad():
    a_next = target_actor_net(next_state)
    q_next = target_critic_net(next_state, a_next)
    q_target = reward + gamma * q_next * (1-done)
criterion = nn.MSELoss()
critic_loss = criterion(q_value, q_target)
```

This model is updated following the same principle as the DQN model, where the MSE between the actual Q-values and the predicted Q-values is used as the loss function for updating the model.

- Explain effects of the discount factor.

From a mathematical perspective, using a discount factor less than one allows us to avoid dealing with infinite sums. Additionally, this discount factor can be interpreted as a way to assign importance to past rewards. A larger discount factor, still less than one, signifies a higher emphasis on past rewards.

- Explain benefits of epsilon-greedy in comparison to greedy action selection.

Epsilon-greedy action selection is advantageous compared to pure greedy selection. Greedy always chooses the best-estimated action, while epsilon-greedy adds exploration by randomly selecting actions sometimes. This helps the model discover potentially better actions, striking a balance between exploiting known actions and exploring new ones. This prevents the model from getting stuck and improves long-term performance in reinforcement learning.

- Explain the necessity of the target network.

Without a target network, the model can only be updated after taking an action. However, with the inclusion of a target network, the model can be updated after a certain number of timesteps. This means that the model takes into account the effects of many actions over time before undergoing an update, resulting in faster convergence during training.

- Describe the tricks used in Breakout and their effects, and how they differ from those used in LunarLander.

1. Fire:
    - Trick: Starting the game after firing.
    - Effect: In Breakout, starting the game after firing is common, and this trick helps in avoiding the consideration of invalid actions before the game begins.
    - Difference from LunarLander: In LunarLander, the game doesn't involve firing. The "Fire" trick is specific to games like Breakout where the action of starting the game is triggered by the "fire" action.
2. Frame Stacking:
    - Trick: Stacking the last 4 consecutive frames.
    - Effect: Stacking frames captures temporal information and motion, which is vital in Breakout due to fast-paced ball and paddle movements.
    - Difference from LunarLander: While stacking frames is used in both Breakout and LunarLander, the dynamics captured in Breakout involve quick changes in object positions. In LunarLander, stacking frames helps in capturing the motion of the lander and its interactions with the terrain.

3. Episodic Life:

    - Trick: Considering an episode done after agent loses a life.
    - Effect: In Breakout, this trick simulates a longer sequence of lives for the agent to learn better survival strategies and maximize rewards over multiple lives.
    - Difference from LunarLander: In LunarLander, the concept of lives or multiple attempts doesn't apply as directly as it does in Breakout. The agent's primary focus in LunarLander is to land the lander safely, and the concept of "episodic life" isn't as relevant.